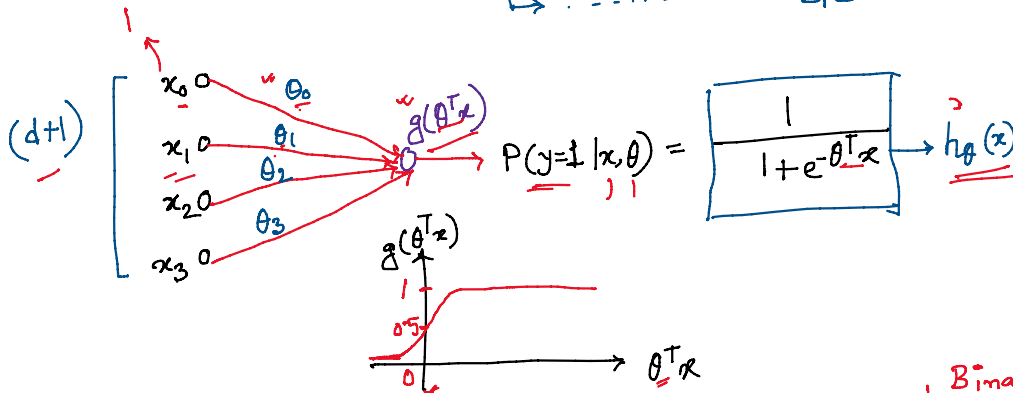
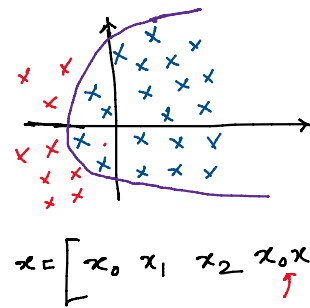
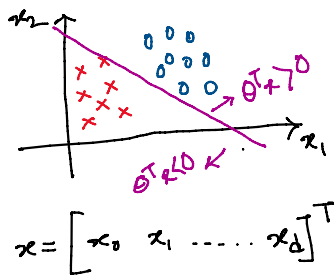


Classification - Logistic regression → benign/malignant tumor cell prediction
→ spam/non-spam email prediction
→ ... etc.



$$J(\theta) = \frac{1}{n} \sum_{i=1}^n [y^i \log(h_\theta(x^i)) + (1-y^i) \log(1-h_\theta(x^i))] \quad \leftarrow \text{Binary cross-entropy}$$

Decision boundary:



Practical considerations:

① Feature scaling:

x_1	x_2	x_3	y
3	10	1200	0
4	30	2100	1
8	20	1000	1
6	40	1100	0
⋮	⋮	⋮	⋮

Since the features have different range in different dimensions, the cost function might get skewed along one of the dimensions & it gets very hard to reach the global minima!

Idea → Converting all the feature dimensions into a single scale without changing the relations among different features.
e.g. by dividing " x_3 " in the table by 1000.

$$\rightarrow -1 \leq x_i \leq 1; 0 \leq x_i \leq 1$$

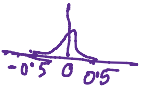
* 2 types of scaling can be used:

① Min-max normalization:

$$x_i \rightarrow \frac{x_i - \min(x)}{\max(x) - \min(x)} \rightarrow 0 < x_i < 1$$

② Standardization:

$$x_i \rightarrow \frac{x_i - \text{mean}(x)}{\text{std}(x)} \rightarrow -0.5 < x_i < 0.5 \text{ (normal distribution)}$$



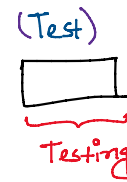
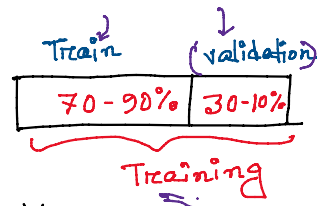
② Hyperparameter selection:

$$\theta = [\theta_0 \ \theta_1 \ \theta_2 \ \dots \ \theta_d]^T \rightarrow \text{Parameters}$$

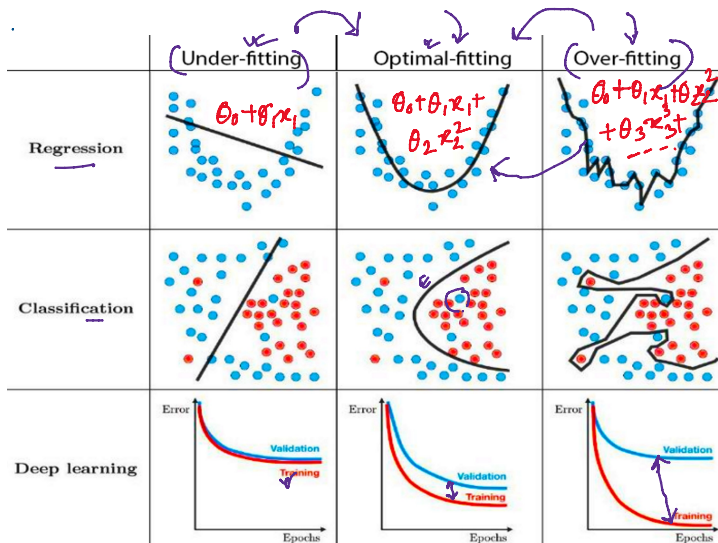
- Which $J(\theta)$?
 - Which features?
 - Which optimizer?
 - How many epochs
- } Hyperparameters

How to select the hyperparameters?

— At the end of each epoch, the model's performance is to be evaluated by the validation data, thus the hyperparameters are tuned!



③ Regularization:



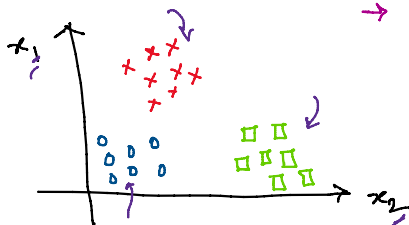
$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n [h_{\theta}(x^{(i)}) - y^{(i)}]^2 + \lambda \sum_{i=1}^n \theta_i^2$$

↓
Regularization term.

Multi-class classification: → Email classification: Friends/Work/Spam

→ image classification: human/cat/dog



$$y = \{0, 1, 2\}$$

*. Algo-1: (One-vs-All) → Extending the idea of logistic regression

(i) ... to predict the probability of

*. Algo-1: (One-vs-All) → Extending the idea of logistic regression

Idea → Train a logistic regression classifier $h_{\theta}^{(i)}$ for each class 'i' to predict the probability of "y=i" → $i=0,1,2$

$$y_{\text{pred}} = \text{arg max}_i h_{\theta}^{(i)}(x)$$

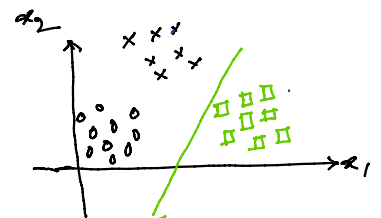
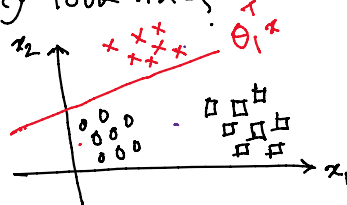
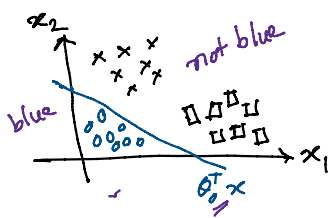
Now,

$$\rightarrow h_{\theta_0}^{(0)}(x) = \frac{1}{1 + e^{-\theta_0^T x}} = P(y=0|x, \theta_0), \theta_0 = [\theta_{00} \ \theta_{01} \ \theta_{02} \ \dots \ \theta_{0d}]^T$$

$$\rightarrow h_{\theta_1}^{(1)}(x) = \frac{1}{1 + e^{-\theta_1^T x}} = P(y=1|x, \theta_1), \theta_1 = [\theta_{10} \ \theta_{11} \ \theta_{12} \ \dots \ \theta_{1d}]^T$$

$$\rightarrow h_{\theta_2}^{(2)}(x) = \frac{1}{1 + e^{-\theta_2^T x}} = P(y=2|x, \theta_2), \theta_2 = [\theta_{20} \ \theta_{21} \ \theta_{22} \ \dots \ \theta_{2d}]^T$$

→ So, how will the decision boundary look like?



$$W = \begin{bmatrix} \theta_0^T \\ \theta_1^T \\ \theta_2^T \end{bmatrix} = \begin{bmatrix} \theta_{00} & \theta_{01} & \dots & \theta_{0d} \\ \theta_{10} & \theta_{11} & \dots & \theta_{1d} \\ \theta_{20} & \theta_{21} & \dots & \theta_{2d} \end{bmatrix}$$

Weight $\downarrow$

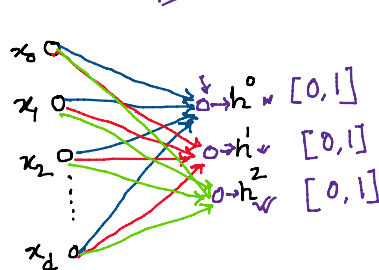
size $\rightarrow 3 \times (d+1)$

weights/params for $h_{\theta}^{(0)}$
weights/params for $h_{\theta}^{(1)}$
weights/params for $h_{\theta}^{(2)}$

So, $h = f(Wx) = f\left(\begin{bmatrix} \theta_0^T x \\ \theta_1^T x \\ \theta_2^T x \end{bmatrix}\right) = \begin{bmatrix} f(\theta_0^T x) \\ f(\theta_1^T x) \\ f(\theta_2^T x) \end{bmatrix} = \begin{bmatrix} \frac{1}{1 + e^{-\theta_0^T x}} \\ \frac{1}{1 + e^{-\theta_1^T x}} \\ \frac{1}{1 + e^{-\theta_2^T x}} \end{bmatrix}$

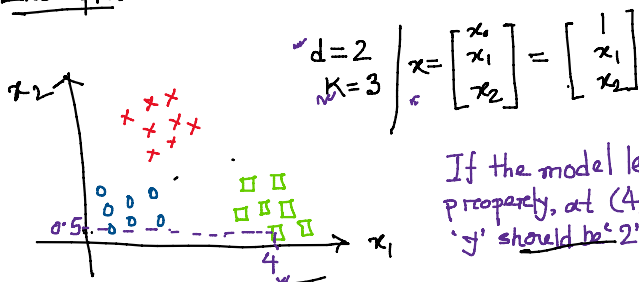
$K \times (d+1)$ $(d+1) \times 1$ $K \times 1$

K -number of classes



$$h = f(Wx) = \begin{bmatrix} h_0 \\ h_1 \\ h_2 \\ \vdots \\ h_K \end{bmatrix}$$

*. Example:



$$d=2 \quad K=3 \quad x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix}$$

If the model learns properly, at (4, 0.5) point 'y' should be '2'.

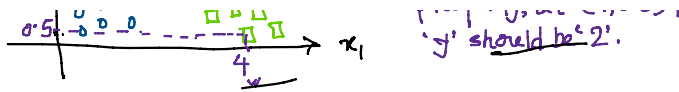
Let's say we have found the best parameters as:

$$\theta_0 = [1, -1, -1]^T, \theta_1 = [-1, -0.5, 1]^T, \theta_2 = [-4, 2, -1]^T$$

$$x = [1, 4, 0.5]^T$$

$$W = \begin{bmatrix} \theta_0^T \\ \theta_1^T \\ \theta_2^T \end{bmatrix} = \begin{bmatrix} 1 & -1 & -1 \\ -1 & -0.5 & 1 \\ -4 & 2 & -1 \end{bmatrix}$$

$$\dots \begin{bmatrix} 1 & -1 & -1 \\ -1 & -0.5 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -3.5 \\ -2.5 \\ 2.5 \end{bmatrix}$$



So, the model will learn by training the three classifiers separately i.e. by minimizing separate cost functions $J^0(\theta_0), J^1(\theta_1), J^2(\theta_2)$

$$\text{Now, } Wx = \begin{bmatrix} 1 & -1 & -1 \\ -1 & -0.5 & 1 \\ -1 & 2 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -3.5 \\ -2.5 \\ 3.5 \end{bmatrix}$$

$$f(Wx) = \begin{bmatrix} \frac{1}{1+e^{-(-3.5)}} \\ \frac{1}{1+e^{-(-2.5)}} \\ \frac{1}{1+e^{-3.5}} \end{bmatrix} = \begin{bmatrix} 0.03 \rightarrow h^0 \\ 0.08 \rightarrow h^1 \\ 0.97 \rightarrow h^2 \end{bmatrix}$$

So, $J_{\text{pred}} = 2$

Problems with "one-vs-all" algorithm:

- (1) Have to train separately (Creates problems in case of datasets with many classes)
- (2) No interpretation for h [$\text{sum}(h) > 1$]

Solⁿ:

Softmax classifier - Extension of Logistic Regression

ideas: ① we want $\text{sum}(h) = 1$

② We want end-to-end learning (single classifier for all classes)

modifications to "one-vs-all":

① Use softmax function $f(\cdot)$ instead of sigmoid function $f(\cdot)$

Say, $Wx = [z_0, z_1, z_2, \dots, z_{k-1}]^T$, softmax function, $f_i = \frac{e^{z_i}}{\sum_{j=0}^{k-1} e^{z_j}}$

$$\text{So, } f(Wx) = \frac{1}{\sum_{j=0}^{k-1} e^{z_j}} \begin{bmatrix} e^{z_0} \\ e^{z_1} \\ e^{z_2} \\ \vdots \\ e^{z_{k-1}} \end{bmatrix}$$

results of using softmax:

- ① $\sum_{i=0}^{k-1} f_i = 1$
- ② $0 \leq f_i \leq 1$

② Change 'y' to one hot vector

$$y = [0, 1, 2] \rightarrow \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}$$

③ Modify the cost function

$$J = -\frac{1}{n} \sum_{i=1}^n \sum_{k=0}^{K-1} y^{(i)}_k \log f_k$$

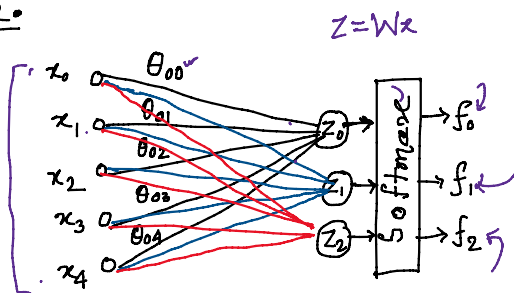
label (True) Prediction of model

Cross Entropy loss

As before, the parameters can be learned using an optimization algorithm like "Gradient Descent"

* Example:

$d=4$
 $K=3$
(4+1)



So,
 $Z_0 = \theta_{00} + \theta_{01}x_1 + \theta_{02}x_2 + \theta_{03}x_3 + \theta_{04}x_4$
 $Z_1 = \theta_{10} + \theta_{11}x_1 + \dots + \theta_{14}x_4$
 $Z_2 = \dots$

Let's say we have got the following parameters:

$\theta_0 = [0, 0.01, -0.05, 0.1, 0.05]^T$

$\theta_1 = [0.2, 0.7, 0.2, 0.05, 0.16]^T$

$\theta_2 = [-0.3, 0.0, -0.45, -0.2, 0.33]^T$

& $x^{(i)} = [1, -15, 22, -44, 56]^T$, $y^{(i)} = 2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

So,
 $Z = Wx = \begin{bmatrix} 0 & 0.01 & -0.05 & 0.1 & 0.05 \\ 0.2 & 0.7 & 0.2 & 0.05 & 0.16 \\ -0.3 & 0.0 & -0.45 & -0.2 & 0.33 \end{bmatrix} \begin{bmatrix} 1 \\ -15 \\ 22 \\ -44 \\ 56 \end{bmatrix} = \begin{bmatrix} -2.58 \\ 0.86 \\ 0.28 \end{bmatrix}_{3 \times 1}$

$f(Wx) = \begin{bmatrix} f_0 \\ f_1 \\ f_2 \end{bmatrix}$

Now, $f_0 = \frac{e^{Z_0}}{e^{Z_0} + e^{Z_1} + e^{Z_2}} = \frac{e^{-2.58}}{e^{-2.58} + e^{0.86} + e^{0.28}} = 0.016$
 Similarly, $f_1 = \frac{e^{Z_1}}{e^{Z_0} + e^{Z_1} + e^{Z_2}} = 0.63$ & $f_2 = \frac{e^{Z_2}}{e^{Z_0} + e^{Z_1} + e^{Z_2}} = 0.353$

So, $y_{pred} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

Now, for i -th sample, $J_i = -\sum_{j=0}^2 y_j^{(i)} \log f_j^{(i)} = -[y_0 \log f_0 + y_1 \log f_1 + y_2 \log f_2]$
 $= -[0 \times \log(0.016) + 0 \times \log(0.63) + 1 \times \log(0.353)]$
 $= 1.04$

natural logarithm
 $\log_e(x)$

Let's say for three other samples, $J_i = 0.08, 1.34, 2.06$

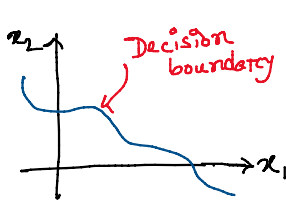
So, $J = \frac{1}{4}(1.04 + 0.08 + 1.34 + 2.06)$

II. Neural Networks (NN):

x_2
Decision boundary

$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_1 x_2 \end{bmatrix}$

In order to learn complex non-linear decision boundaries, we need to introduce interaction terms / polynomial terms / complex



$$x = \begin{bmatrix} x_1 \\ x_2 \\ \rightarrow x_1 x_2 \\ \rightarrow \sin(x_1 + x_2) \\ \vdots \end{bmatrix}$$

decision boundaries, we need to introduce product terms / exponential terms / complex functions in the feature space

This creates problems for large scaled datasets, since the number of parameters also increases proportionally with the complexity of the feature space.

↓
Solution → Neural networks!!

→ NNs are universal function approximators

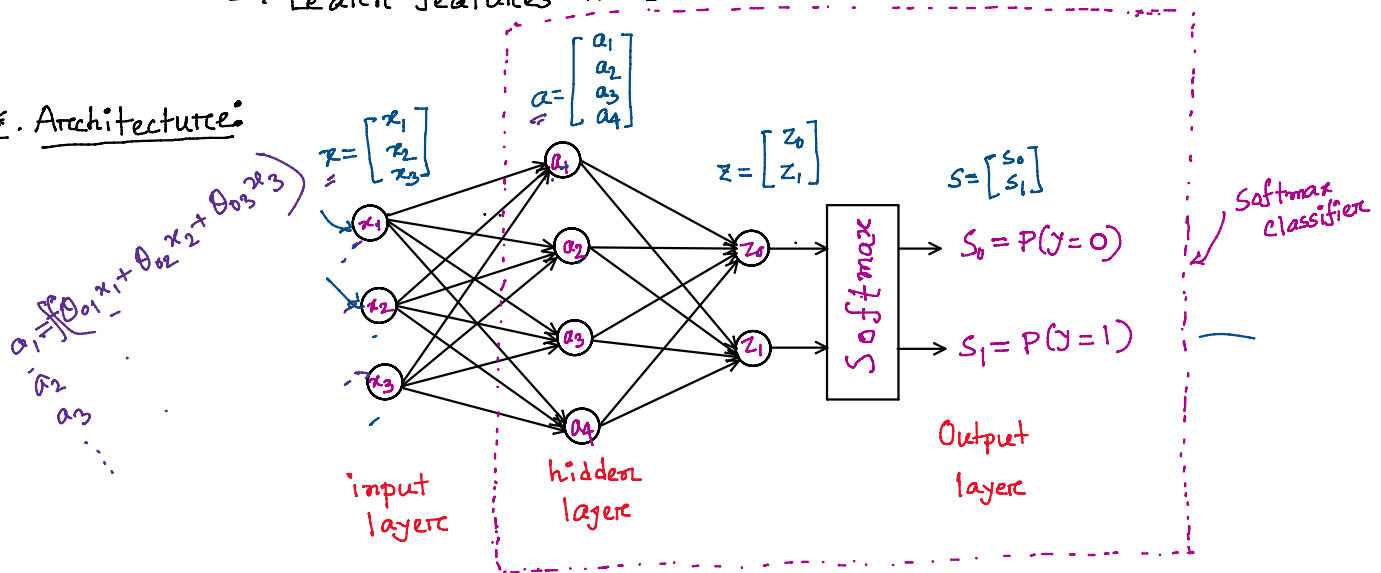
[Given $x_0, x_1, x_2, \dots$ they can learn functions like $x_0^2, x_1 x_2, \sin(x_1 + x_2)$]

→ Instead of increasing feature length, depth is increased (Adding more hidden layers)

→ Deep neural networks can learn more complex functions

→ Learn features themselves

*. Architecture:

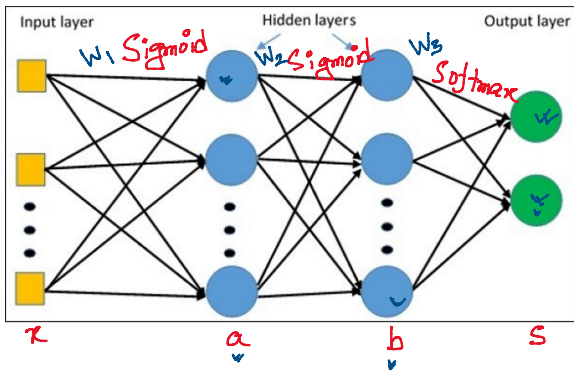


Calculations:

$$\begin{aligned} a &= f(W_1 x) \\ z &= (W_2 a) \\ s &= f'(z) \end{aligned}$$

f : some activation function (sigmoid)
 f' : softmax

*. Multi-layer architecture:



$$a = g(w_1 x)$$

$$b = g(w_2 a)$$

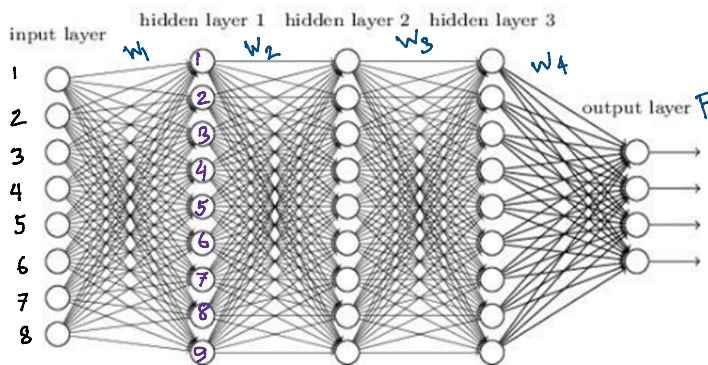
$$s = f(w_3 b)$$

$g = \text{sigmoid / any other non-linear activation}$

$f = \text{softmax}$

*. Training deep neural networks:

Deep neural network



$$\begin{aligned} h_1 &= g(w_1 x) \\ h_2 &= g(w_2 h_1) \\ h_3 &= g(w_3 h_2) \\ z &= w_4 h_3 \\ F &= f(z) \end{aligned}$$

9x1 ← 9x8 ← 8x1

final feature vector

$$\text{cost function, } J(w_1, w_2, w_3, w_4) = \frac{1}{n} \sum_{i=1}^n \left[- \sum_{k=0}^{K-1} y_k \log f_k \right] \leftarrow \text{cross-entropy loss}$$

Deep neural networks are also

known as:

- artificial neural network
- feed forward neural network
- fully connected neural network
- Dense neural network

Gradient descent:

$$w_1^{(k+1)} = w_1^k - \nabla_{w_1} J$$

$$w_2^{(k+1)} = w_2^k - \nabla_{w_2} J$$

⋮

z is a function of w_4 . So, $\frac{\partial J}{\partial w_4}$ can be calculated directly

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial w_4} \times \frac{\partial w_4}{\partial w_3} \times \frac{\partial w_3}{\partial w_2} \times \frac{\partial w_2}{\partial w_1}$$

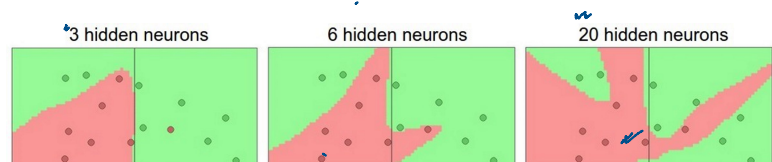
$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial w_4} \times \frac{\partial w_4}{\partial w_3} \times \frac{\partial w_3}{\partial w_2}$$

$$\frac{\partial J}{\partial w_3} = \frac{\partial J}{\partial w_4} \times \frac{\partial w_4}{\partial w_3}$$

(Chain rule)

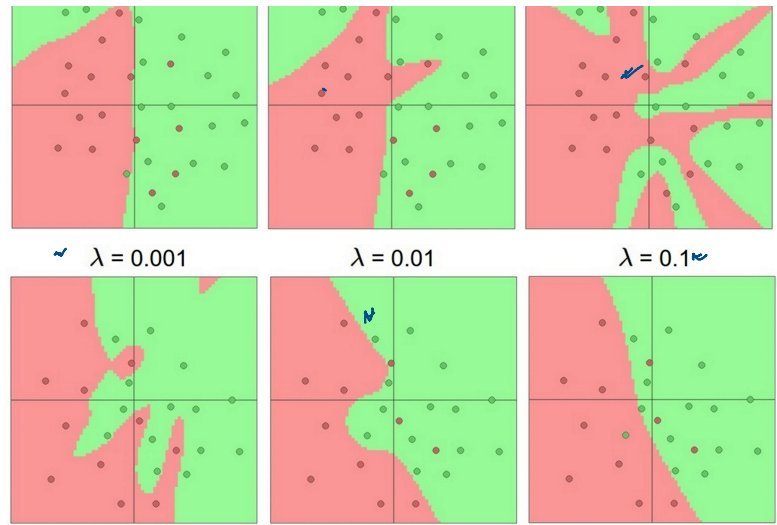
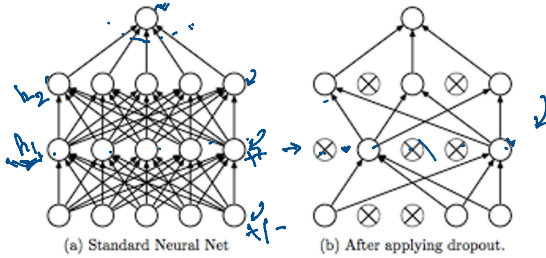
*. Practical considerations:

① Number of neurons



* Practical considerations:

- ① Number of neurons
- ② Overfitting → Regularization
→ Dropout



③ Activation function:

* sigmoid → $g(z) = \frac{1}{1 + e^{-z}}$

* tanh → $g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

* ReLU → $g(z) = \begin{cases} z & ; z > 0 \\ 0 & ; \text{otherwise} \end{cases}$

